

P3769R0

Clarification of placement new deallocation

Published Proposal, 2025-06-22

Author:

[Lauri Vasama](#)

Audience:

EWG, CWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Current draft:

vasama.github.io/wg21/D3769.html

Current draft source:

github.com/vasama/wg21/blob/main/D3769.bs

Abstract

Clarifies wording for selection of deallocation functions in placement new expressions.

Table of Contents

1 Introduction

2 Implementation divergence

3 Wording

3.1 [expr.new]

References

Informative References

§ 1. Introduction

[P3492R2] was forwarded to CWG in Hagenberg. In addition to its main language additions, it provided a drive-by fix for the poorly specified selection of deallocation functions in placement new expressions. The wording changes were split into this paper to make it easier to review the changes happening concurrently in this area:

1. The wording improvements now separated into this paper.
2. The language changes in [P3492R2] (Sized deallocation for placement new).
3. The language changes in [P2719R5] (Type-aware allocation and deallocation).

In addition it is proposed that the wording changes in this paper be applied as a defect report.

§ 2. Implementation divergence

There are two cases of implementation divergence around the use of deallocation functions in placement new expressions:

1. Whether function templates are considered valid candidates and argument deduction is performed. The current standard says no, however EDG alone conforms, and only in the case of global deallocation function templates.
2. The current wording specifies that if a deleted or inaccessible deallocation function is selected, the program is ill-formed. GCC and EDG are non-conforming in this case.

```
template<int>
struct alloc {};

void* operator new(size_t, alloc<0>& a);

template<int I>
void operator delete(void*, alloc<I>& a); // #1

void operator delete(void*, alloc<0>& a) = delete; // #2

int main() {
    alloc<0> al;
    new (al) thrower();
}
```

Current implementation behaviour:

	As shown	With #1 removed	With #2 removed
Clang	Warning: ambiguity	Error: #2 is deleted	Uses #1
GCC	—	—	Uses #1
MSVC	Error: #2 is deleted	Error: #2 is deleted	Uses #1
EDG	—	—	— ¹
P3769	—	Error: #2 is deleted	Uses #1

¹ If the operators are class specific, EDG selects an unambiguous function template (i.e. with #1 removed) only if it is not deleted. If the selected function is deleted, no diagnostic is issued.

§ 3. Wording

§ 3.1. [expr.new]

Modify [expr.new] as follows:

If the *new-expression* does not begin with a unary `::` operator and the allocated type is a class type `T` or an array thereof, a search is performed for the deallocation function's name in the scope of `T`. Otherwise, or if nothing is found, the deallocation function's name is looked up by searching for it in the global scope. The set of lookup results are the *candidates* used in the placement deallocation function selection process.

A placement deallocation function is selected as follows:

- In each case where a candidate is a function template, candidate function template specializations are generated using template argument deduction ([temp.over], [temp.deduct]). The arguments used for the deduction are the first argument followed by the additional arguments.
- Eliminate from further consideration any functions whose *parameter-declaration-clause* terminates with an ellipsis, but that of the allocation function does not.
- Eliminate from further consideration any functions whose *parameter-declaration-clause* does not terminate with an ellipsis, but that of the allocation function does.
- Eliminate from further consideration any functions where the number of parameters is not equal to the number of parameters of the selected allocation function.
- Eliminate from further consideration any functions where, after parameter transformations ([dcl.fct]), the types of the function parameters are not identical to the types of the parameters of the allocation function, except for the first parameter of each.
- If exactly one function remains, that function is selected and the selection process terminates.
- Otherwise, the selection process terminates without having selected a deallocation function.

[Note: A deallocation function with an additional trailing parameter compared to the allocation function is never matched, even if a default argument is provided. —end note]

A declaration of a placement deallocation function matches the declaration of a placement allocation function if it has the same number of parameters and, after parameter transformations ([dcl.fct]), all parameter types except the first are identical. If the lookup finds a single matching deallocation function, that function will be called; otherwise, no deallocation function will be called. If the lookup finds a usual deallocation function and that function, considered as a placement deallocation function, would have been selected as a match for the allocation function, the program is ill-formed. For a non-placement allocation function, the normal deallocation function lookup is used to find the matching deallocation function ([expr.delete]). In any case, the matching deallocation function (if any) shall be non-deleted and accessible from the point where the *new-expression* appears.

The following examples are entirely new, but are not highlighted in green in order to improve readability:

[Example:

```
struct A {};  
struct T {};  
  
void* operator new(std::size_t s, A& al); // #1  
  
template<int = 0>  
void operator delete(void* p, A& al); // #2  
  
A al;  
new (al) T(); // Uses #1 and #2.
```

—end example]

[Example:

```
template<int I>  
struct A {};  
struct T {};  
  
void* operator new(std::size_t s, A& al); // #1  
  
template<int I>  
void operator delete(void* p, A<I>& al);  
void operator delete(void* p, A<0>& al);  
  
A<0> al;  
new (al) T(); // Uses #1
```

—end example]

[Example:

```
struct A {};  
struct T {};  
  
void* operator new(std::size_t s, A& al);  
void operator delete(void* p, A& al) = delete;  
  
A al;  
new (al) T(); // error: attempted to use deleted function
```

—end example]

§ References

§ Informative References

[P2719R5]

Louis Dionne, Oliver Hunt. *Type-aware allocation and deallocation functions*. 12 May 2025.

URL: <https://wg21.link/p2719r5>

[P3492R2]

Lauri Vasama. *Sized deallocation for placement new*. 17 February 2025. URL:

<https://wg21.link/p3492r2>